

DSP Viva Preparation

Natural answers, practical explanations, and experiment follow-ups

A practical question-and-answer guide based on your DSP lab experiments and syllabus.

How to use this guide

Give the meaning first, explain how it works, and connect it to your experiment. Learn the reasoning so you can explain it in your own words.

Topics

1. Introduction to DSP
2. Basic signals
3. Sampling and aliasing
4. DTFT and DFT
5. DFT properties
6. FFT, DIT, and DIF
7. Linear and circular convolution
8. FIR filters and window techniques
9. Butterworth IIR filters
10. Hilbert transform

Explain any experiment

Includes the full revised answers from the conversation. Notation: $\hat{x}$ means the Hilbert transform of x ; f_s is the sampling frequency.

1. Introduction to DSP

What is digital signal processing?

Digital signal processing means analysing or modifying a signal after representing it as numerical samples. For example, if we record speech, we can process those samples to reduce noise, identify frequencies, or compress the recording. So DSP helps us extract useful information from a signal or change it for a particular purpose.

Why do we prefer digital processing?

The main advantage is flexibility. We can change the processing by changing the program, rather than rebuilding a circuit. Digital processing also gives repeatable results, makes signals easy to store, and allows complicated algorithms to be implemented. However, its accuracy still depends on sampling, quantisation, and numerical precision.

Where is DSP used?

DSP is used in speech and music processing, mobile communication, medical instruments, image processing, radar, and industrial monitoring. For example, in an ECG system, digital filters can reduce interference and make the heart's electrical activity easier to examine.

What happens in a complete DSP system?

First, an analog signal passes through an anti-aliasing filter. An ADC then converts it into digital samples. A processor performs the required operation, such as filtering. If we need an analog output, a DAC converts the processed samples back, followed by a reconstruction filter.

What is the difference between discrete-time and digital signals?

A discrete-time signal is defined only at particular time instants, but its amplitude can still take any value. A digital signal also has its amplitude represented with a finite number of levels. Sampling makes time discrete, while quantisation makes amplitude discrete.

2. Basic signals

Why do we study basic signals?

Basic signals help us understand more complicated signals and test how systems behave. An impulse tells us the impulse response of a system, a step shows how it responds to a sudden change, and a sinusoid helps us study its behaviour at a particular frequency.

What is a unit impulse?

In discrete time, a unit impulse has value one at $n = 0$ and zero everywhere else. In continuous time, the impulse is an ideal signal concentrated at one instant with unit area. We cannot generate that ideal continuous impulse exactly, so in our experiment we approximate it with a narrow pulse.

Why was the pulse height 20 and its width 0.05 seconds?

An impulse should have unit area. The rectangular approximation has area equal to height multiplied by width, so $20 \times 0.05 = 1$. If we make the pulse narrower while keeping its area one, its height must increase.

Does MATLAB actually generate a continuous-time signal?

MATLAB stores a finite set of sample values, so it does not represent every possible time instant. We use a very small time step and connect the samples with a plot, which makes the waveform look continuous.

Why do we use stem instead of plot for a discrete sequence?

A stem plot shows each sample separately, which makes the discrete nature of the sequence clear. A normal plot connects neighbouring samples and can give the impression that values exist between them.

3. Sampling and aliasing

What is the sampling theorem?

The sampling theorem tells us how fast we must sample a band-limited signal to preserve its information. For reliable reconstruction of a general signal, the sampling frequency should be greater than twice its highest frequency component. This keeps the repeated copies of its spectrum from overlapping.

$$f_s > 2f_{\max}$$

What is aliasing?

Aliasing happens when different analog frequencies become indistinguishable after sampling. As a result, a high-frequency signal may appear to be a lower-frequency signal. Once that ambiguity has occurred, we generally cannot recover the original frequency from the samples alone.

Why did the 50 Hz signal appear as 10 Hz when sampled at 60 Hz?

At a sampling rate of 60 Hz, the Nyquist frequency is only 30 Hz, so the 50 Hz component lies outside the usual unambiguous range. Its samples are identical to those of a 10 Hz cosine. That is why the reconstructed signal appears at $60 - 50 = 10$ Hz.

What is the difference between Nyquist rate and Nyquist frequency?

The Nyquist rate is twice the highest frequency present in the input signal. The Nyquist frequency is half the sampling frequency. So one is determined by the signal, while the other is determined by how we sample it.

Can we sample exactly at twice the signal frequency?

That is a boundary case, so I would not consider it reliable for a general signal. For example, a sine wave sampled exactly twice per cycle may be sampled only at its zero crossings, giving all-zero samples. Our experiment uses a cosine, which gives alternating positive and negative samples. In practice, we sample above the boundary to allow a safety margin.

How do we prevent aliasing?

We limit the input bandwidth using an analog low-pass filter before sampling, and then choose a sufficiently high sampling rate. The filter removes frequencies that would otherwise fold into the useful frequency range.

Can we remove aliasing using a digital filter afterward?

Generally, no. After aliasing, the unwanted component has already become indistinguishable from a valid lower-frequency component. A digital filter may remove that frequency, but it would also remove any genuine signal present there.

Why is there reconstruction error even when the sampling rate is sufficient?

The sampling theorem assumes ideal reconstruction using all the required samples. Our program uses only a finite record, so the sinc interpolation is truncated. This introduces error, especially near the ends of the record, which is why we measure error in the middle portion.

4. DTFT and DFT

What is the DTFT?

The DTFT describes the frequency content of a discrete-time sequence. Although the signal is discrete in time, the DTFT is a continuous function of frequency. It tells us how much of each frequency is present, and its spectrum repeats every 2π radians per sample.

What is the DFT?

The DFT converts a finite block of time-domain samples into a finite set of frequency-domain coefficients. Each coefficient corresponds to a particular frequency and describes its magnitude and phase contribution. It is useful because a computer can directly calculate and store these coefficients.

$$X[k] = \sum_{n=0}^{N-1} x[n] \exp(-j2\pi kn/N)$$

How are DTFT and DFT related?

For a finite sequence, the DFT gives equally spaced samples of its DTFT. The DTFT describes the spectrum continuously, while the DFT evaluates it at a fixed number of frequency points.

Can we calculate more than an 8-point DFT?

Yes. Eight points is just the length selected in our experiment. We can calculate a 16-point, 32-point, 100-point, or any other positive integer-length DFT. The limitation is mainly the available computation and memory.

Does the number of points have to be a power of two?

No. The DFT itself has no power-of-two restriction. That restriction applies to a pure radix-2 FFT algorithm. Other FFT algorithms can handle lengths that are not powers of two.

What if we have eight samples but request a 16-point DFT?

We can append eight zeros and calculate the 16-point DFT. This gives more closely spaced frequency samples of the same finite-record spectrum. It does not give us additional measured information.

Is a larger DFT more complex?

Yes, in terms of computation. A direct DFT requires roughly N^2 operations, so increasing the length makes it much more expensive. We normally use FFT algorithms for larger transforms because they reduce the computation to the order of $N \log N$.

Why is the DFT output complex even when the input is real?

The DFT compares the input with complex sinusoids, which contain both cosine and sine parts. We need both parts to represent phase as well as magnitude. A real input therefore generally produces complex coefficients, but those coefficients have conjugate symmetry.

What does $X[0]$ mean?

$X[0]$ is the DC coefficient, and it equals the sum of the input samples. In our experiment, the samples from one to eight add up to 36. Dividing that coefficient by the number of samples gives the mean, which is 4.5.

What is a frequency bin?

A frequency bin is one of the frequency points evaluated by the DFT. Adjacent bins are separated by f_s/N hertz. For example, if the sampling rate is 800 Hz and we calculate an eight-point DFT, the bin spacing is 100 Hz.

Why are there negative frequencies?

Negative frequencies arise naturally when we represent signals using complex exponentials. A real sinusoid can be expressed as a combination of positive- and negative-frequency components. In the DFT of a real signal, these components appear as conjugate pairs.

What is conjugate symmetry?

For a real input, the negative-frequency coefficient is the complex conjugate of the corresponding positive-frequency coefficient. Their magnitudes are equal and their phases have opposite signs. In our experiment, bins one and seven demonstrate this relationship.

Does zero-padding improve frequency resolution?

It makes the frequency grid finer, so peaks can be displayed and located more conveniently. But it does not improve our underlying ability to separate two closely spaced frequencies. For that, we generally need a longer observation of the actual signal.

What is spectral leakage?

When we analyse only a finite portion of a signal, its frequency content can spread into neighbouring bins. This is especially visible when the record does not contain an integer number of sinusoidal cycles. Applying a suitable window reduces sidelobes, although it also broadens the main peak.

Why did your direct DFT and MATLAB FFT differ slightly?

They calculate the same mathematical transform, but use different orders of numerical operations. Computers have finite precision, so small roundoff differences occur. An error around 10^{-14} indicates very close agreement.

5. DFT properties

What does the linearity property mean?

If we add two sequences, their DFTs also add. Similarly, multiplying a sequence by a constant multiplies its DFT by the same constant. This allows us to analyse a complicated signal as a sum of simpler components.

What happens if we circularly shift a sequence?

A circular time shift changes the phase of the DFT coefficients but leaves their magnitudes unchanged. The amount of phase change depends on the bin number and the size of the shift.

What is the convolution property of DFT?

Multiplication of two DFTs corresponds to circular convolution of their sequences. To obtain linear convolution through this method, we must first zero-pad the sequences sufficiently.

What is Parseval's theorem?

Parseval's theorem says that the energy calculated in the time domain equals the energy calculated in the frequency domain, with the appropriate scaling. The transform changes the representation of the signal, but preserves its energy information.

$$\sum_{n=0}^{N-1} |x[n]|^2 = (1/N) \sum_{k=0}^{N-1} |X[k]|^2$$

6. FFT, DIT, and DIF

What is FFT?

FFT stands for fast Fourier transform. It is an efficient algorithm for calculating the DFT. It does not produce a different transform; it obtains the same result while avoiding repeated calculations.

How does it reduce the computation?

It divides a large transform into smaller transforms and combines their results. The symmetry and periodicity of the complex exponential factors allow many calculations to be reused.

What is a twiddle factor?

A twiddle factor is a complex exponential used in the DFT and FFT. It rotates a value by a particular phase angle. In an FFT, these rotations allow the smaller transform results to be combined correctly.

$$W_N^k = \exp(-j2\pi k/N)$$

What is radix-2 FFT?

A radix-2 FFT repeatedly divides the transform into two smaller transforms. To continue this division down to the smallest operations, the transform length must be a power of two.

What is decimation-in-time?

In DIT, we separate the input samples into even-indexed and odd-indexed groups. We calculate smaller DFTs for those groups and combine them using twiddle factors. It is called decimation-in-time because the decomposition starts with the time-domain indices.

What is decimation-in-frequency?

In DIF, we first form sums and differences of input samples separated by half the transform length. These calculations lead to smaller transforms associated with even and odd frequency bins. The decomposition is therefore organised around the frequency-domain output.

What is a butterfly?

A butterfly is the basic repeated computation in an FFT. It takes two values and combines them using addition, subtraction, and a twiddle factor. The name comes from the shape of its signal-flow diagram.

$$\text{DIT butterfly: } u = a + Wb; v = a - Wb$$

How do DIT and DIF differ practically?

In a conventional iterative implementation, DIT commonly takes bit-reversed input and produces output in natural order. DIF commonly takes natural-order input and produces bit-reversed output. Their twiddle factors also appear at different positions in the butterflies, but both calculate the same DFT.

How many stages are needed for an eight-point radix-2 FFT?

Three stages, because $\log_2 8 = 3$. Each stage has four butterflies, so the complete calculation uses twelve butterflies.

What is bit reversal?

We write an index in binary and reverse the order of its bits. For an eight-point FFT, index three is 011; reversing it gives 110, which is six. This rearrangement supports the pattern of the iterative FFT calculations.

7. Linear and circular convolution

What is convolution?

Convolution tells us how an input signal interacts with a system's impulse response. Each input sample produces a shifted and scaled copy of the impulse response, and we add those copies to obtain the output. That is why convolution is fundamental to filtering.

What is an impulse response?

It is the output of a system when we apply a unit impulse. For a linear time-invariant system, knowing the impulse response allows us to calculate the output for any input using convolution.

What is the difference between linear and circular convolution?

In linear convolution, samples shift normally and the output extends over the full combined duration. In circular convolution, the indices wrap around after N samples. Contributions that go beyond the end are added back at the beginning.

Why is the linear convolution length $L_x + L_h - 1$?

The first input sample starts the first shifted impulse response, while the final input sample starts the last one. The complete overlap therefore extends from index zero to $L_x + L_h - 2$, giving $L_x + L_h - 1$ output samples.

Explain the circular convolution result in your experiment.

Our linear convolution was [8, 22, 43, 52, 45, 28]. For a four-point circular convolution, the last two values wrap around. So 45 is added to 8, and 28 is added to 22, giving [53, 50, 43, 52].

How can circular convolution give the same result as linear convolution?

We zero-pad both sequences to a common length of at least $L_x + L_h - 1$. Then the complete linear convolution fits within the block, so no nonzero tail wraps around.

Why use FFT to calculate convolution?

For long sequences, direct convolution can be expensive. We can transform both sequences, multiply their spectra, and take the inverse transform. With suitable zero-padding, this produces linear convolution efficiently.

8. FIR filters and window techniques

What is an FIR filter?

An FIR filter has a finite-duration impulse response. Its output is a weighted sum of the present input sample and a fixed number of previous samples. In the usual implementation, it does not feed previous outputs back into the calculation.

What is filter order?

For an FIR filter, the order is one less than the number of coefficients. Our filter has order 60, so it contains 61 coefficients, or taps.

What is the difference between FIR and IIR filters?

An FIR filter has a finite impulse response, while an IIR filter generally has a response that continues indefinitely because of feedback. FIR filters with finite coefficients are stable and can be designed for exact linear phase. IIR filters usually meet similar magnitude requirements with fewer coefficients, but their stability and phase response require more care.

What is linear phase?

Linear phase means phase changes linearly with frequency. This produces constant group delay, so the frequency components passed by the filter are delayed by the same amount. It helps preserve their relative timing.

Why is linear phase desirable?

A signal's shape depends on how its frequency components line up. If different components experience different delays, their alignment changes and phase distortion occurs. Linear phase avoids that particular distortion, although the filter can still change the waveform by altering component amplitudes.

Are all FIR filters linear phase?

No. A real FIR filter has generalised linear phase when its coefficients are symmetric or antisymmetric about the midpoint. Simply having a finite impulse response does not guarantee linear phase.

What is the delay of your filter?

Our symmetric FIR has order 60, so its group delay is $60/2 = 30$ samples. At a sampling rate of 1000 Hz, each sample represents one millisecond, so the delay is 30 milliseconds.

Why can't we implement an ideal low-pass filter directly?

An ideal low-pass filter has an abrupt cutoff, but its impulse response extends infinitely in both time directions. A practical causal filter cannot use infinitely many samples or future inputs. We therefore approximate the ideal response using a finite filter.

How does the window method work?

We start with the ideal filter's impulse response, retain a finite portion, and multiply it by a window. We also shift the response to make the finite implementation causal. The chosen window controls how the truncation affects the frequency response.

Why do we need a window instead of simply cutting the response?

Simply cutting the response is equivalent to using a rectangular window. That abrupt truncation produces strong sidelobes and ripple. A smoother window reduces the abrupt change at the ends, improving sidelobe suppression, usually at the cost of a wider transition band.

What is the tradeoff between rectangular and Blackman windows?

For the same length, the rectangular window gives a narrower main lobe but higher sidelobes. Blackman gives stronger sidelobe suppression but a wider main lobe. So Blackman is useful when rejection is important, while a sharper transition may require a longer filter.

Which window is best?

There is no universally best window. We choose based on the required transition width, attenuation, and available filter order. In our experiment, Blackman gave the highest measured stopband attenuation, but that comes with a transition-width tradeoff.

What happens when we increase FIR order?

The transition generally becomes sharper for the same window family. However, we need more calculations and memory, and the linear-phase delay increases. So increasing order improves some aspects of the response but also increases the implementation cost.

What is the Gibbs phenomenon?

It is the oscillation that appears near a discontinuity when we approximate it using a truncated series or impulse response. With rectangular truncation, increasing the length makes the oscillations narrower around the discontinuity, but does not eliminate the limiting peak overshoot.

Explain the cutoff in your experiment.

Our passband edge was 200 Hz and stopband edge was 300 Hz. We selected their midpoint, 250 Hz, as the design cutoff. Since MATLAB's normalised cutoff is measured relative to the Nyquist frequency, we used $250/500 = 0.5$.

Why did we use even order for the high-pass FIR?

A symmetric FIR with odd order has even length and a forced zero at the Nyquist frequency. That is unsuitable for a normal high-pass filter, which should pass frequencies near Nyquist. Using even order avoids that restriction.

What does freqz show?

It calculates how the filter responds to different frequencies. The magnitude tells us how much each frequency is amplified or attenuated, and the phase tells us the phase shift introduced by the filter.

9. Butterworth IIR filters

What is a Butterworth filter?

A Butterworth filter is designed to have a maximally flat magnitude response in the passband. Its response changes smoothly and monotonically, without ripple. It is useful when a smooth magnitude response is more important than a very sharp transition at a low order.

If it has no ripple, why do we specify R_p ?

In this design, R_p represents the maximum allowed passband loss. It does not mean the Butterworth response oscillates. For example, specifying one decibel means the gain at the passband edge must remain within that allowed loss.

Why does an IIR response continue indefinitely?

Previous outputs are fed back into later calculations. Even after the input stops, the stored state can continue producing an output. In a stable filter, this response gradually decays.

What determines IIR stability?

For a causal rational filter, the uncanceled poles must lie strictly inside the unit circle. Their locations determine whether the natural response decays, persists, or grows.

What do `buttord` and `butter` do?

`buttord` finds the minimum order and a suitable critical frequency that meet our requirements. We then pass those results to `butter`, which calculates the filter coefficients.

What is the bilinear transformation?

It converts an analog filter design into a digital filter design. It maps stable analog poles into the stable region of the digital plane. However, the frequency mapping is nonlinear, so it introduces frequency warping.

What is prewarping?

Prewarping adjusts the analog design frequencies before applying the bilinear transformation. This ensures that the important frequency edges end up at the intended positions in the digital filter.

Explain your filtering result.

Our input contained 500 Hz and 3600 Hz tones. The low-pass filter mainly retained the 500 Hz tone and attenuated the 3600 Hz tone. The high-pass filter did the opposite. Both designs had order six and met the specified passband and stopband limits.

Why does the beginning of the output look different?

The filter starts from its initial state, usually zero. It takes some time for that startup response to settle, so the beginning contains a transient as well as the response to the input.

10. Hilbert transform

What is the Hilbert transform?

The Hilbert transform produces a quadrature version of a signal. Ideally, it shifts positive-frequency components by minus 90 degrees and negative-frequency components by plus 90 degrees, while preserving their magnitudes. For example, the Hilbert transform of a cosine is a sine.

How does it work?

We can think of it as a filter with a particular frequency response. It multiplies positive-frequency components by $-j$ and negative-frequency components by $+j$. Those multiplications rotate the components by minus and plus 90 degrees respectively.

Is it just a time delay?

For one sinusoid, a 90-degree phase shift can be represented as a quarter-period delay. But for a signal containing several frequencies, a fixed time delay produces a different phase shift at each frequency. A Hilbert transform provides the quadrature shift across the frequency components instead.

What is an analytic signal?

We form an analytic signal by combining the original real signal with its Hilbert transform as the imaginary part. This complex representation contains the original signal and its quadrature component. Ideally, its spectrum has no negative-frequency content.

$$z(t) = x(t) + j \hat{x}(t)$$

What does MATLAB's hilbert function return?

It returns the complex analytic signal, rather than only the Hilbert transform. The real part is the original signal, and the imaginary part is its Hilbert transform. That is why our code uses `imag(hilbert(x))`.

How can FFT be used to form the analytic signal?

We take the FFT of a real sequence, double the positive-frequency coefficients, remove the negative-frequency coefficients, and retain DC. For an even-length sequence, we also retain the Nyquist coefficient. Taking the inverse FFT then gives the analytic signal.

Where is the Hilbert transform used?

It is used for quadrature signal generation, envelope detection, instantaneous phase analysis, and single-sideband modulation. These applications use the relationship between the original signal and its 90-degree-shifted version.

How does it help with envelope detection?

The magnitude of the analytic signal combines the original and quadrature components. For a suitable narrowband amplitude-modulated signal, this follows the amplitude envelope. For a simple unit-amplitude cosine, the analytic signal's magnitude is one.

$$|z(t)| = \sqrt{x^2(t) + [\hat{x}(t)]^2}$$

How does it help generate single-sideband modulation?

We combine two modulation paths: one uses the original signal with a cosine carrier, and the other uses its Hilbert transform with a sine carrier. With the appropriate addition or subtraction, one sideband cancels and the other remains.

What happens if we apply the Hilbert transform twice?

For the applicable non-DC components, applying it twice gives the negative of the original signal. Each application contributes another quadrature rotation, so the total is equivalent to 180 degrees. DC is an exception because the Hilbert transform removes it.

Explain your experiment naturally.

We generated a 50 Hz cosine sampled at 1000 Hz. We then used hilbert to obtain its analytic signal and took the imaginary part. That result matched a 50 Hz sine, which confirms the quadrature relationship. The small error, around 10^{-14} , was due to numerical roundoff.

Why was the numerical result so accurate?

We used a one-second record containing exactly 50 cycles. The waveform therefore fits neatly into the periodic block assumed by FFT-based processing. This avoids a boundary discontinuity and helps produce a very accurate result.

Explain any experiment

The aim was to _____. We used _____ because _____. The output showed _____, which agrees with the theory. If we change _____, I would expect _____.

Example: FIR experiment

The aim was to compare low-pass and high-pass FIR filters designed using different windows. We used the window method to obtain a finite approximation to the ideal impulse response. The responses showed that smoother windows generally improved stopband suppression, with a transition-width tradeoff. If we increase the order, we can usually obtain a sharper transition, but the computation and delay also increase.

That level of explanation shows both theoretical understanding and practical awareness.